

C++ / DIRECTX 12

PROGRAM EXPLANATION

2.5Dステージエディター 設計・実装資料

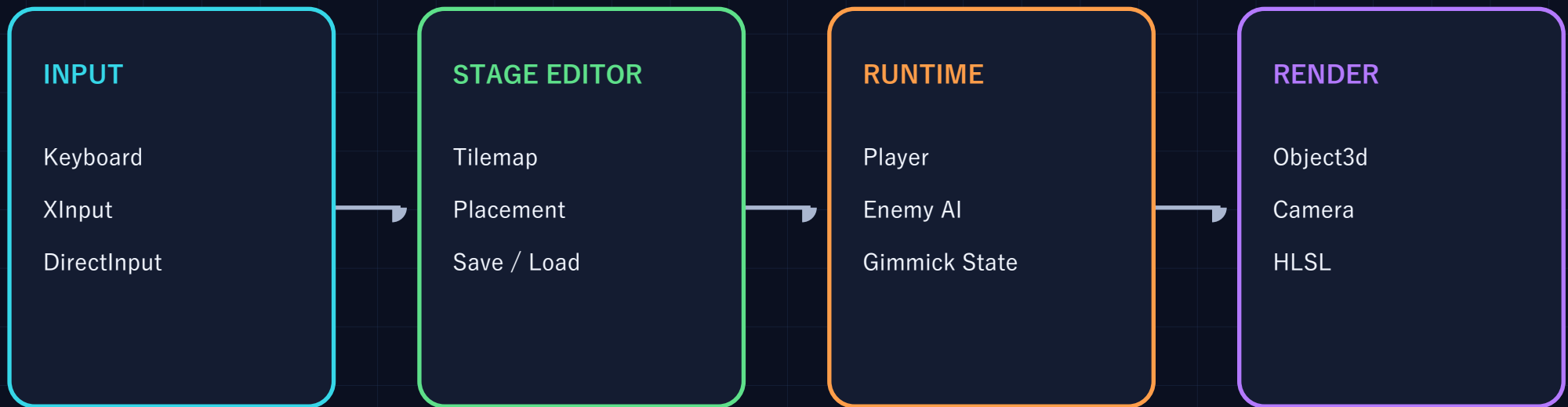


LE3C_10 サクラ タイキ

C++ / DirectX 12 / ImGui / HLSL

▼ Models
axis.obj
plane.obj
terrain.obj
bunny.obj
fence.obj
AnimatedCube.glTF
human/walk.glTF

システム構成



編集データは StageEditor が保持し、GamePlay時のみ実行時状態を更新。描画・入力・カメラは既存エンジン層へ接続する。

データモデルと保存形式

Placement

- id / item_id
- position / rotation / scale
- variant
- move_offset

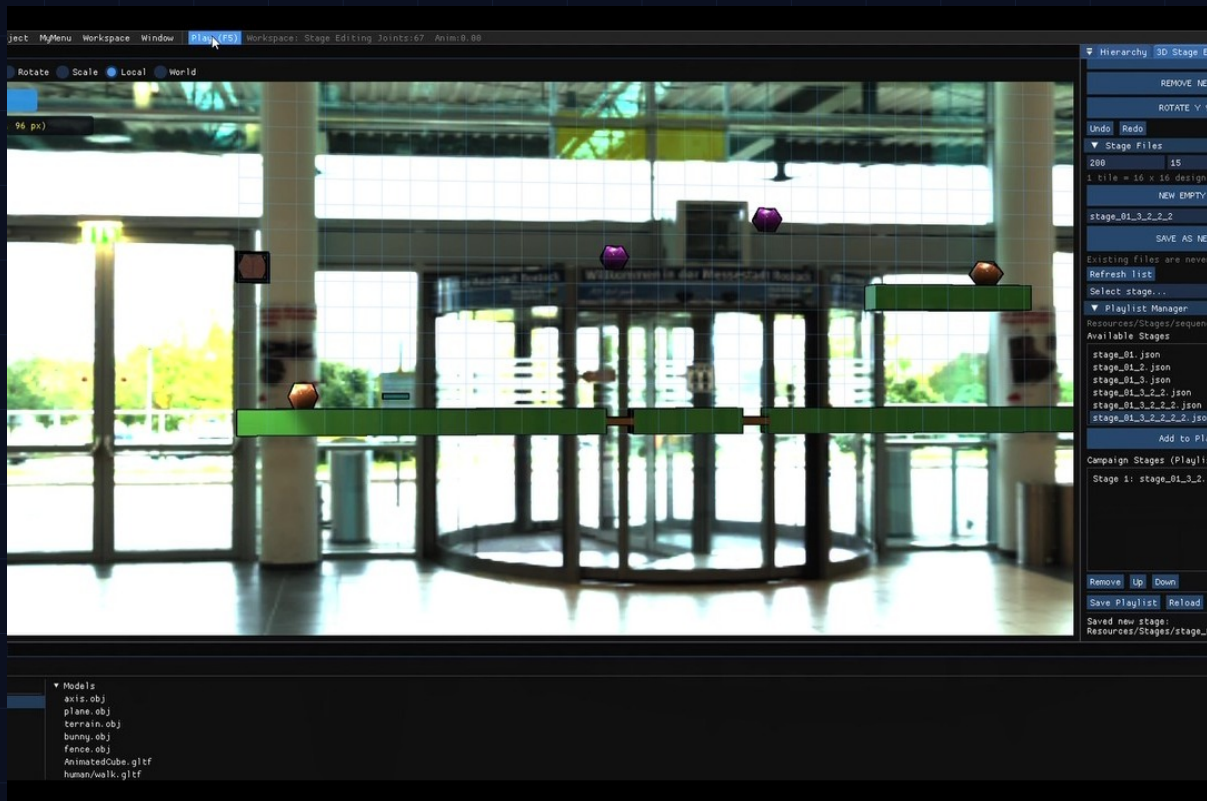
Stage JSON v5

- tile_size = 16
- size = width x height
- tiles[][]
- objects[]

StageEditor.cpp / SaveAsNewStage

```
root["version"] = 5;
root["tile_size"] = kTileSizePixels;
root["tiles"] = json::array();
root["objects"].push_back({
    {"id", p.id}, {"item_id", p.itemId},
    {"position", {p.position.x, ...}},
    {"variant", p.variant},
    {"move_offset", {p.moveOffset.x, ...}}
});
```

16 x 16基準のグリッド編集



座標変換

GridToWorld

```
world.x = (gridX + 0.5f)
          * tileWorldSize;
world.y = (gridY + 0.5f)
          * tileWorldSize;
```

- 画面上のマウス位置をGridへ変換
- タイルと自由配置OBJを分離
- Undo / RedoはSnapshotで復元

プレイヤー移動と当たり判定

- AABBでタイル・配置物との重なりを判定
- X軸とY軸を分けて解決し、壁抜けを抑制
- 下方向衝突でGroundedを確定
- 落下・敵・トゲ接触時はチェックポイントへ復帰
- ダメージ後1秒のクールダウンで連続接触バグを防止

StageEditor.cpp / UpdatePlayer

```
playerVelocity.y -= gravity * dt;  
MovePlayerHorizontal(velocity.x * dt);  
MovePlayerVertical(velocity.y * dt);  
UpdateGimmickCollisions(input, dt);  
ResolvePlacedSolidCollisions();  
  
if (playerDamageCooldown_ <= 0.0f)  
    RespawnPlayer(true);
```

3種類の敵AI



WALKER

床を左右に巡回
壁または崖を検知すると反転



FLYER

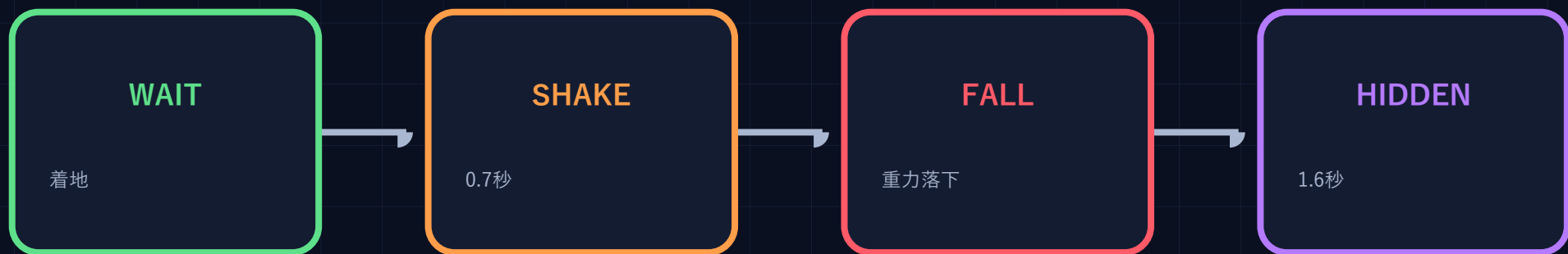
初期位置を中心に
sin波で上下左右へ周期移動



CHASER

7.0以内で追跡開始
9.0以上で解除し初期位置へ復帰

ギミックを状態遷移で管理



復帰時は保存済み配置座標へ戻し、実行時位置・速度・タイマーだけを初期化する。

横スクロールカメラと入力統合



- 自機位置を右方向へ追従
- 指数補間で滑らかに移動
- ステージ端でClampし、範囲外を表示しない
- WASD / 矢印 / Enter / Space
- XInputとDirectInputを同一APIへ統合

Compute Shader Skinning

- 頂点・影響情報・パレットをStructuredBufferで入力
- Compute Shaderで頂点位置と法線を変換
- 出力頂点をUAVへ保存
- 同一形状の複数描画やShadow処理へ再利用可能
- Dispatch対象数をSkinCluster側で管理

Skinning.CS.hlsl / SkinCluster.cpp

```
[numthreads(1024, 1, 1)]
void main(uint3 id : SV_DispatchThreadID) {
    if (id.x >= gNumVertices) return;
    // weighted palette transforms
    gOutputVertices[id.x] = skinned;
}

skinCluster.computeDispatchRequired = true;
```

設計方針

- 編集データと実行時状態を分離する
- 各機能をモード単位で有効化する
- 配置・保存・読込・試遊までを一つのループにする
- 不具合を再現条件から切り分け、共通処理で再発を防ぐ